Outillage Formel

LOS

Chapitre 2

Détails du Chapitre 2

- Introduction
- Langages des données
- Langage pour la dynamique

Introduction

- Un programme est muni de 2 concepts
  - Syntaxe : séquence de mots du langage de programmation
  - Sémantique : relative au sens du programme, ce que le programme fait ou l'action qu'il réalise (a trait au comportement)
  - **Exemple :**
    - Syntaxe : z:= x; x:= y; y:=z : formé de 3 instructions séparées par des ;
    - Sémantique : échanger des variables x et y en utilisant une variable intermédiaire
  - Pourquoi la sémantique?
    - Afin de permettre de cerner très précisément ce que réalise le programme
    - Afin de réaliser des traducteurs (compilateurs) et des optimiseurs
    - Afin de pouvoir appliquer des méthodes formelles de vérification
  - Ce qui garantit la production de logiciels sûrs
    - Transports, Télécommunications, Transactions commerciales, ….

Introduction

- La sémantique peut être vue comme une fonction :

$$sem : programme \rightarrow sens$$

- Dans toute la suite, on distinguera :
  - Les données et structures de données manipulées par le programme (entiers, …)
    - Représentent le vecteur d'état d'un programme, le contexte, l'environnement d'exécution ou l'état de mémoire, …..
    - Permettent donc de décrire les états du système
    - La formalisation du contexte nécessite l'introduction d'outils formels.
  - Les différentes opérations permettant de calculer le résultat d'un programme

Introduction

➔ Dans toute la suite, on distinguera :

- Les données et structures de données manipulées par le programme (entiers, …)
- Les différentes opérations permettant de calculer le résultat d'un programme

Cette vision n'est plus valable pour les programmes dans lesquels la terminaison n'est pas requise, dans ce cas, un programme peut être vu comme un calcul représentant la suite des états d'un programme

Langage des Données

Langage des données

Ensembles

- Notations :
  - A, B, C, … des ensembles
  - a, b, c, … des éléments d'ensembles
  - Ensemble vide : ø, Ensemble singleton : {a}
  - Opérations ensemblistes : voir Tableau 1 ci-dessous

Langage des données

Opérateur	Syntaxe	Condition	Définition
Union	$A \cup B$	$A \subseteq S \ \wedge B \subseteq S$	$\{a \mid a \in S \ \wedge (a \in A \ \vee \ a \in B)\}$
Intersection	$A \cap B$	$A \subseteq S \ \wedge B \subseteq S$	$\{a \mid a \in S \ \wedge (a \in A \ \wedge \ a \in B)\}$
Différence	$A - B$	$A \subseteq S \ \wedge B \subseteq S$	$\{a \mid a \in S \ \wedge (a \in A \ \wedge \ a \notin B)\}$
Ensemble des parties	$\mathbb{P}(A)$		Puissance de A (2^A)
Ensemble des parties non vides	$\mathbb{P}_1(A)$		$\mathbb{P}(A) - \{\emptyset\}$
Ensemble des parties finies	$\mathbb{F}(A)$		
Ensembles des parties finies non vide	$\mathbb{F}_1(A)$		$\mathbb{F}(A) - \{\emptyset\}$

Langage des données

➔ Ensembles

- Exemple :
  - $A = \{1, 2, 4\}$ *et* $B = \{2,\ 4,\ 6\ , 8\}$
  - $A - B = \{1\}$
  - $\mathbb{P}(A) = \{\emptyset, \{1\}, \{2\}, \{4\}, \{1,2\}, \{1,4\}, \{2,4\}, \{1,2,4\}\}$
- Représentation : 2 manières
  - En extension : $A = \{1, 2, 4\}$
  - En compréhension : $A = \{a \in S \mid P(a)\}$ où P(a) désigne un prédicat ou une propriété.
- Paire ordonnée : $x \mapsto y$ qui signifie la paire (x, y), notée aussi <x, y>
- Produit cartésien : $A \times B = \{a \longmapsto b \mid a \in A \ \wedge \ b \ \in B\}$

$$A = \{1, 2\} \quad B = \{3\} \quad A \times B = \{1 \longmapsto 3, 2 \ \longmapsto 3\}$$

➔ λ.Abstraction / λ.Notation

$$\lambda x.(x \in s \mid E) = \{x, y \mid x, y \in s \times t \wedge y = E\}$$

$$\lambda x.(x \in s \wedge P \mid E) = \{x, y \mid x, y \in s \times t \wedge P \wedge y = E\}$$

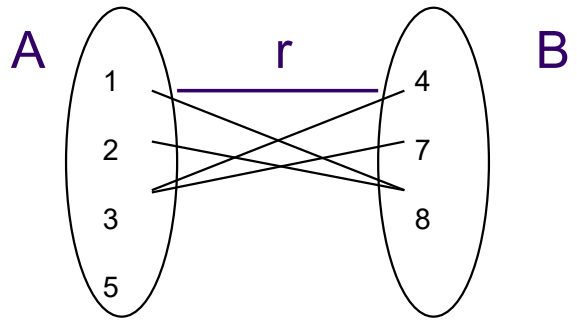
<u>Exemple :</u> $\lambda x.(x \in 0 \cdot\cdot 3 \mid 3 - x) = \{ 0 \mapsto 3, 1 \mapsto 2, 2 \mapsto 1, 3 \mapsto 0 \}$

Langage des données

Relations

- Une relation de A vers B est un sous ensemble du produit cartésien A x B. Les éléments de cette relation sont notés $a \longmapsto b$ où $a \in A$ et $b \in B$,
  - b est **l'image** de a , a est l'image réciproque (**antécédent**) de b
  - $A = \{1, 2, 4\}$ *et* $B = \{2,\ 4,\ 6\ , 8\}$
  - $A - B = \{1\}$
  - $\mathbb{P}(A) = \{\emptyset, \{1\}, \{2\}, \{4\}, \{1,2\}, \{1,4\}, \{2,4\}, \{1,2,4\}\}$

- L'ensemble de toutes les relations entre A et B, noté $A \longleftrightarrow B$, est tel que : $A \longleftrightarrow B = \mathbb{P}(A \times B)$



$$\boldsymbol{r \in A \longleftrightarrow B = \{1 \longmapsto 4,\quad 1 \longmapsto 8,\quad 2 \longmapsto 8,\quad 3 \longmapsto 4,\quad 3 \longmapsto 7\}}$$

Langage des données

Relations

- Une relation R est :
  - Réflexive sur A si $\forall a, aRa$
  - Symétrique si $\forall (a,b) \in A,\ aRb \Longrightarrow bRa$
  - Anti symétrique si $\forall (a,b) \in A, (aRb \wedge bRa) \Longrightarrow a = b$
  - Transitive $\forall (a,b,c) \in A, (aRb \wedge bRc) \Longrightarrow aRc$

    (1) + (2) + (4) : R est une relation d'équivalence

    (1) + (3) + (4) : R est une relation d'ordre
- Ordre Total : $\forall (a,b) \in A,\ a\ R\ b\ ou\ b\ R\ a$ sinon : ordre partiel
- Relation inverse r^{-1} : $r^{-1} = \{a \longmapsto b \mid b \longmapsto a\ \in r\}$
- Domaine de r : $r \in A \longleftrightarrow B \quad dom(r) = \{a \mid a \in A \wedge \exists b\,.(b \in B\ \wedge a \longmapsto b \in r\}$
- Codomaine (range) de r : $ran(r) = \{b \mid b \in B \wedge \exists a\,.(a \in A\ \wedge a \longmapsto b \in r\}$
- Composition des relations :

$$r \in A \longleftrightarrow B\ \wedge\ q \in B \longleftrightarrow C$$

$$r\,;q = q\ \mathbf{o}\ r = \{a,c \mid a \longmapsto c \in A \times C\ \wedge\ \exists b\,.(\,b \in B\ \wedge a \longmapsto b\ \in r\ \wedge b \longmapsto c\ \in q\}$$

- Identité : $Id(A) = \{a,b \mid a \longmapsto b\ \in A \times A\ \wedge a = b\}$

Langage des données

Relations

- **Opérations avancées** :
  - Restriction sur le domaine :

$$r \in A \leftrightarrow B \text{ et } E \subseteq A,\ E \lhd r = \{a, b \mid a \mapsto b \in r \wedge a \in E\}$$

  - Restriction sur le co-domaine :

$$r \in A \leftrightarrow B \text{ et } F \subseteq B,\ r \rhd F = \{a, b \mid a \mapsto b \in r \wedge b \in F\}$$

- Exemple :

$$r = \{1 \mapsto a, 2 \mapsto b, 2 \mapsto c, 3 \mapsto a, 3 \mapsto c\}$$
$$dom(r) = \{1, 2, 3\}$$
$$ran(r) = \{a, b, c\}$$
$$\{1,3\} \lhd r = \{1 \mapsto a, 3 \mapsto a, 3 \mapsto c\}$$
$$r \rhd \{c\} = \{2 \mapsto c, 3 \mapsto c\}$$

Voir **l'Annexe** jointe au Poly pour plus de détails sur l'ensemble des opérations utilisées dans ce cours.

Langage des données

➔ Fonctions

- Une fonction f de A vers B est une relation pour laquelle chaque élément du domaine possède au plus une image, c'est-à-dire :

$$\forall \boldsymbol{a_1}, \boldsymbol{a_2} \in \boldsymbol{A}, \qquad \boldsymbol{si\ a_1 = a_2\ \ alors\ \ f(a_1) = f(a_2)}$$

- Type de fonction :
  - Fonction partielle :

$$A \nrightarrow B = \{f \mid f \in A \leftrightarrow B \ \wedge \ \forall a, b, c\,.\,((a,b) \in f \ \wedge (a,c) \in f \ \Longrightarrow b = c)\}$$
$$= \{f \mid f \in A \leftrightarrow B \ \wedge (f^{-1}; f) \subset Id(B)\}$$

  - Fonction Totale : fonction partielle où tous les éléments de A possèdent une image dans B

$$A \rightarrow B = \{f \mid f \in A \nrightarrow B \ \wedge dom(f) = A\}$$

  - Injective : $A \rightarrowtail B = f \mid f \in A \nrightarrow B \ \wedge f^{-1} \in B \nrightarrow A$: Chaque élément de B possède au plus un antécédent dans A.
  - Injective totale : $A \rightarrowtail B$ = $A \rightarrowtail B \quad A \rightarrow B$: tous les élément de A sont impliqués dans la fonction
  - Surjective, bijective, ….. : voir A**nnexe.**

Langage des données

→ Opérations

- Une opération * sur l'ensemble A est une application (fonction totale) de A x A dans A.
- Une opération * peut être
  - Commutative : a * b = b * a
  - Associative : (a * b) * c = a * (b * c)
  - Admet un élément neutre à droite si ∀a, a * e = a et à gauche si ∀a, e * a = a
  - Admet un élément absorbant à droite si ∀a, a * x = x et à gauche si ∀x, x * a = x
  - Un élément a pour inverse à droite si ∀a, a * a' = e et à gauche si ∀a, a' * a = e
  - Un élément a est idempotent si ∀a, a * a =a
  - Une opération * est idempotente si tout élément de A est idempotent par cette opération

Langage des données

➔ Morphismes

- Soit les ensembles :
  - E munit d'une fonction f, d'une opération * et d'une relation R
  - E' munit d'une fonction f', d'une opération *' et d'une relation R'
- Le quadruplet noté <E, f, *, R> (resp. <E', f', *', R'>) est une structure.
- Un morphisme de <E, f, *, R> vers <E', f', *', R'> est une fonction de E vers E' telle que si

$$(x, y, z) \in E, \quad (x', y', z') \in E' \quad sont\ tels\ que\ x' = \varphi(x),\ y' = \varphi(y),\ z' = \varphi(z)$$

- Alors on a :
  - Préservation de la fonction : si y = f(x) alors y' = f'(x'), i.e., $\varphi(y) = f'(\varphi(x))$
  - Préservation de l'opération : si z = x * y alors z' = x' *' y', i.e., $\varphi(z) = \varphi(x) *' \varphi(y)$
  - Préservation de la relation : si x R y alors x' R' y', i.e., $\varphi(x)\ R'\ \varphi(y)$
- Un isomorphisme est un morphisme bijectif.
- Deux structures sont isomorphes si elles sont reliées par un isomorphisme.

Cette notion est utilisée pour transposer une représentation (par exemple, un programme) dans une autre représentation sur laquelle il est possible de raisonner

Langage des données

Logique des propositions

- P et Q deux propositions logiques. Le tableau suivant résume les différents connecteurs logiques :

Symbole	Nom
$\neg$	Non
$\wedge$	Et
$\vee$	Ou
$\Rightarrow$	Implique
$\Leftrightarrow$	Equivaut
$\forall$	Quelque soit
$\exists$	Il existe

Langage des données

➔ Logique des propositions

- La formule $\forall x, P(x) \Rightarrow Q(x)$: signifie que tout x vérifiant P vérifie aussi Q. Si aucun x ne vérifie P alors $\forall x, P(x) \Rightarrow Q(x)$: est vraie.

- De nombreuses propriétés logiques peuvent s'écrire sous forme d'équivalence logique :

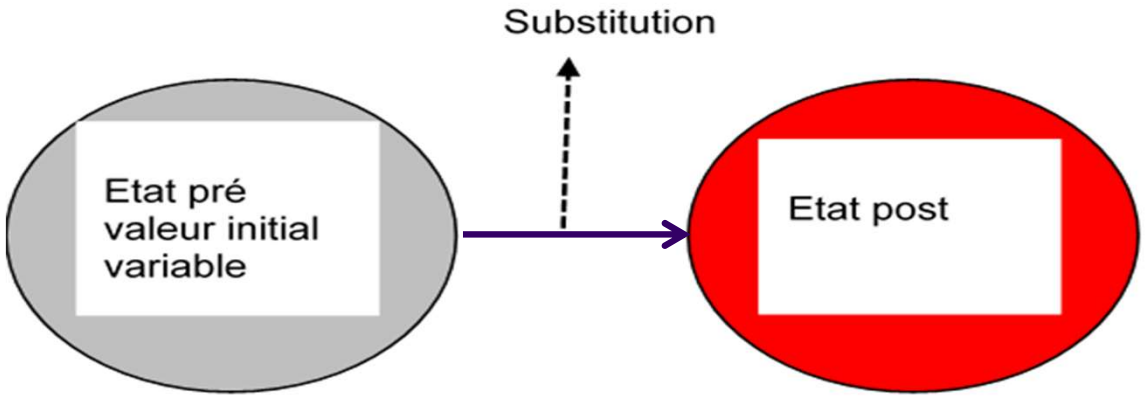
$$P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$$
$$P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$$
$$\neg\neg P \Leftrightarrow P$$
$$P \Longrightarrow Q \Leftrightarrow P \vee Q, \qquad P \Longrightarrow Faux \Leftrightarrow P$$
$$\neg(P \vee Q) \Leftrightarrow P \wedge Q \ , \ \neg(P \wedge Q) \Leftrightarrow P \vee Q$$

Langage pour la Dynamique

Langage pour la dynamique

- C'est un langage de spécification et de description des opérations : Substitutions Généralisées (SG). Ces substitutions sont issues de la théorie des transformateurs des prédicats.
- Le langage SG est généralement utilisé pour décrire les changements d'états. En effet, une substitution est un moyen de représenter une transition.

Logiciel (ou machine) :



Etat : est représenté par une combinaison particulière des valeurs de ses variables.

Langage pour la dynamique

➔ ***Substitutions SG***

- *Ne sont pas nécessairement déterministes : une SG spécifie un ensemble de transitions potentielles.*
- *Lors de l'implantation, une seule transition sera effectivement réalisée (peu importe laquelle)*
- *L'indéterminisme est appréciable quand il s'agit de spécifier ou de raffiner*

➔ On distingue un ensemble de substitutions primitives à partir des quelles d'autres constructeurs sont définis.

➔ Dans une notation mathématique, les substitutions primitives se résument comme suit (avec e, f : expression; p : prédicat; S,T : substitutions) :

Langage pour la dynamique

Notation	Nom
$x := e$	Substitution simple
$x, y := e, f$	Substitution multiple simple
Skip (*)	Substitution sans effet
P|S	Substitution pré conditionnée
$P \Rightarrow S$	Substitution gardée
S ▯ T	Substitution choix borné
@z.S	Substitution choix non bornée
S ;T	Séquencement
While P do S Invariant I variant e end(#)	Substitution itération

(*) : aboutit à un état Post identique à l'état Pré

(#) : **Invariant I** : un prédicat : propriété toujours vérifiée par l'ensemble des variables décrivant le système.

Variant e : une expression

Langage pour la dynamique

➔ Typologie des SG

- **P|S** : permet de donner des contraintes sur les entrées (variables d'état et paramètres). L'opération ne peut être invoquée que dans sa précondition. (en dehors de P, le logiciel ou système est non prédictible)

  Exemple : ***PRE :*** $x \in IMPAIR\ \ THEN\ y := \frac{x-1}{2} \quad END$

- $P \Longrightarrow S$ **:** permet de donner des hypothèses sous lesquelles la substitution S sera utilisée (si P n'est pas vérifié, la substitution n'est pas exécutée, elle est prohibée dans les implémentations)

  Exemple : $SELECT\ x > y\ THEN\ d := \mathrm{x.y}\ \mathrm{END}$

- **S □ T** : permet de décrire des spécifications non déterministes **: x := 1 □ x := 2** décrit l'affectation d'une des valeurs (1 ou 2) à la variable x. Le programmeur est libre d'implanter l'une ou l'autre substitution ou encore selon le contexte.

Langage pour la dynamique

➔ Typologie des SG

- **@z.S** : étend le non déterminisme

  Exemple : ***@z.(z < x ⇒ x := z)*** *a pour effet d'affecter à la variable x une valeur strictement supérieure à la précédente.*

- **Substitution iteration**:

  WHILE P do S INVARIANT I VARIANT e END

Correspond à une exécution itérative de S tant que le prédicat P est satisfait.

- Le variant e :

  (i) une variable entière qui se trouve initialisée à la première exécution ;

  (ii) décroit ensuite strictement à chaque itération de la boucle ;

  (iii) garantit la terminaison de la boucle et interdit par conséquent les boucles infinies.

Langage pour la dynamique

→ Notations syntaxique des SG

Syntaxe (pseudo programme)	Définition
BEGIN S END (Bloc)	S
PRE P THEN S END (Pré-conditionnée)	P|S
IF P THEN S ELSE T END	$P \Longrightarrow S \;\square\; (\neg P \Rightarrow T)$
IF P THEN S END	IF P THEN S ELSE Skip END
$\boldsymbol{x} := \boldsymbol{E} \parallel \boldsymbol{y} := \boldsymbol{F}$ (symbole de parallélisme, substitution multiple)	x, y := E, F
Choice S OR …. OR T END	S □ …. □ T
ANY x WHERE P THEN S END	@x.(P ⇒ S)
$\boldsymbol{x} :\in \boldsymbol{U}$ (choix dans un ensemble)	ANY y WHERE y ∈ U THEN x:=y END
Let x BE x = E IN S END	ANY x WHERE x = E THEN S END

Langage pour la dynamique

➔ Sémantique des SG

- La notation **[x := e] R** (avec e une expression et R un prédicat) : traduit la substitution uniforme de x par e dans R (sachant que x est une variable libre dans R

- ***Remarque :*** *x est une variable non libre (ou liée) dans un prédicat P :*

  Si elle n'est pas présente dans P,

  ou

  Elle est seulement présente dans la portée de certains quantificateurs.

  Dans l'expression $\forall\, n. (n \in \mathbb{N}\ et\ n > x)$ *:* x est libre, n et y sont liés

Langage pour la dynamique

→ Sémantique des SG

Cas de substitution	Réduction	Condition
[x, y := e, f] R	[z := f][x := e][y := z] R	z | e, f, R (z n'est pas libre dans e,f ,R)
[Skip] R	R	
[P|S] R	$P \wedge [S]\, R$	
[S ▯ T] R	$[S]R \wedge [T]R$	
[P ⇒ S]R	P ⇒ [S] R	
[@z.S] R	$\forall z\,.\,[S]R$	Z\R
[S ; T] R	[S] ([T] R)	

Langage pour la dynamique

➔ Sémantique des SG : Exemples

- Substitution multiple

$$[x, y := y, x]\ (x < y)$$
$$[z := x][x := y][y := z]\ (x < y)$$
$$[z := x][x := y]\ (x < z)$$
$$[z := x]\ (y < z)$$
$$(y < x)$$

- Substitution pré-conditionnée :

$$[x > 0 \mid x := x-1]\ (-1 \leq x \leq 1)$$
$$(x > 0) \wedge [x := x-1]\ (-1 \leq x \leq 1)$$
$$(x > 0) \wedge (-1 \leq x-1 \leq 1)$$
$$(x > 0) \wedge (0 \leq x \leq 2)$$
$$(0 < x \leq 2)$$

- Substitution choix non borné : $[@z\ .\ (z > 0 \Rightarrow x := x+z)]\ (x > 3)$

$$\forall z\ .\ (z > 0 \Rightarrow [x := x+z]\ (x > 3)$$
$$\forall z\ .\ (z > 0 \Rightarrow x+z > 3)$$

Langage pour la dynamique

➔ Sémantique des SG : Cas de la boucle

- [WHILE P DO S INVARIANT I VARIANT V END] R

Équivaut :

I	-- l'invariant est vrai avant la boucle
$\forall x\,.\,(I \wedge P \implies [S]I)$	-- l'invariant est conservé dans la boucle
$\forall x\,.\,(I \wedge P \implies V \in \mathbb{N})$	-- le variant est un entier naturel
$\forall x\,.\,(I \wedge P \wedge [n := V] \implies [S](V < n))$	-- le variant décroit à chaque pas
$\forall x\,.\,(I \wedge P) \implies R)$	-- la sortie de la boucle implique R

<u>Remarque :</u> Une des difficultés de programmer avec les boucles est d'exhiber **l'invariant** et de justifier sa **terminaison.**

Langage pour la dynamique

➔ Syntaxe des opérations

Paramètres de sortie ← nom opérations (paramètres d'entrées) =

G

;

- Les paramètres entrées/sorties sont optionnels
- Le passage de paramètre se fait par valeur
- Dans une opération, la substitution G est en général une substitution pré-conditionnée

Langage pour la dynamique

➔ Syntaxe des opérations

- **Exemple 1** : division euclidienne
  - */*qq et rr correspondent au quotient et au reste */*
  - */*aa et bb qui correspondent au dividende et au diviseur */*

MACHINE
DIVISION
OPERATIONS
qq , rr ← divi(aa,bb) =
PRE
$aa \in NAT \ \wedge bb \in NAT1$
THEN
ANY *ss, tt* ***WHERE***
$ss \in NAT \ \wedge tt \in NAT \wedge aa = bb.ss + tt \ \wedge tt < bb$
THEN
qq, rr := ss, tt
END
END;

Langage pour la dynamique

➔ Syntaxe des opérations

- **Exemple 2** : Un ascenseur qui a une porte à chaque étage et on ne distingue pas les appels intérieurs de ceux extérieurs ; pas de pannes.
- Modélisation des ensembles et de l'état :

Langage pour la dynamique

SETS

MODE = {stop, moving}
-- état de l'ascenseur

CONSTANTS

max_floors, FLOORS

PROPERTIES

max_floors ∈ NAT1 ∧ FLOORS = 0..max_floors -- ensemble d'étages

VARIABLES

openf , callf , pos, mode

INVARIANT

openf ⊆ FLOORS ∧ -- ensemble portes ouvertes

callf ⊆ FLOORS ∧ -- ensemble d'appels faits

pos ∈ FLOORS ∧ -- position courante

mode ∈ MODE -- mode courant